

Sequence Container

序列式容器

vector

`std::vector` 是 STL 提供的 **内存连续的、可变长度** 的数组（亦称列表）数据结构。能够提供线性复杂度的插入和删除，以及常数复杂度的随机访问。

为什么要使用 vector

作为 Oler，对程序效率的追求远比对工程级别的稳定性要高得多，而 `vector` 由于其对内存的动态处理，时间效率在部分情况下低于静态数组，并且在 OJ 服务器不一定开全优化的情况下更加糟糕。所以在正常存储数据的时候，通常不选择 `vector`。下面给出几个 `vector` 优秀的特性，在需要用到这些特性的情况下，`vector` 能给我们带来很大的帮助。

vector 可以动态分配内存

很多时候我们不能提前开好那么大的空间（eg：预处理 1~n 中所有数的约数）。尽管我们能知道数据总量在空间允许的级别，但是单份数据还可能非常大，这种时候我们就需要 `vector` 来把内存占用量控制在合适的范围内。`vector` 还支持动态扩容，在内存非常紧张的时候这个特性就能派上用场了。

vector 重写了比较运算符及赋值运算符

`vector` 重载了六个比较运算符，以字典序实现，这使得我们可以方便的判断两个容器是否相等（复杂度与容器大小成线性关系）。例如可以利用 `vector<tchar>` 实现字符串比较（当然，还是用 `std::string` 会更快更方便）。另外 `vector` 也重载了赋值运算符，使得数组拷贝更加方便。

vector 便利的初始化

由于 `vector` 重载了 `=` 运算符，所以我们可以方便的初始化。此外从 C++11 起 `vector` 还支持 [列表初始化](#)，例如 `vector<LTint> data {1, 2, 3};`。

vector 的使用方法

以下介绍常用用法，详细内容 [请参见 C++ 文档](#)。

构造函数

用例参见如下代码（假设你已经 `using` 了 `std` 命名空间相关类型）：

```

1 // 1. 创建空vector; 常数复杂度
2 vector<int> v0;
3 // 1+. 这句代码可以使得向vector中插入前3个元素时, 保证常数时间复杂度
4 v0.reserve(3);
5 // 2. 创建一个初始空间为3的vector, 其元素的默认值是0; 线性复杂度
6 vector<int> v1(3);
7 // 3. 创建一个初始空间为3的vector, 其元素的默认值是2; 线性复杂度
8 vector<int> v2(3, 2);
9 // 4. 创建一个初始空间为3的vector, 其元素的默认值是1,
10 // 并且使用v2的空间配置器; 线性复杂度
11 vector<int> v3(3, 1, v2.get_allocator());
12 // 5. 创建一个v2的拷贝vector v4, 其内容元素和v2一样; 线性复杂度
13 vector<int> v4(v2);
14 // 6. 创建一个v4的拷贝vector v5, 其内容是{v4[1], v4[2]}; 线性复杂度
15 vector<int> v5(v4.begin() + 1, v4.begin() + 3);
16 // 7. 移动v2到新创建的vector v6, 不发生拷贝; 常数复杂度; 需要 C++11
17 vector<int> v6(std::move(v2)); // 或者 v6 = std::move(v2);

```

► 测试代码

可以利用上述的方法构造一个 `vector`, 足够我们使用了。

元素访问

`vector` 提供了如下几种方法进行元素访问

1. `at()`

`v.at(pos)` 返回容器中下标为 `pos` 的引用。如果数组越界抛出 `std::out_of_range` 类型的异常。

2. `operator[]`

`v[pos]` 返回容器中下标为 `pos` 的引用。不执行越界检查。

3. `front()`

`v.front()` 返回首元素的引用。

4. `back()`

`v.back()` 返回末尾元素的引用。

5. `data()`

`v.data()` 返回指向数组第一个元素的指针。

迭代器

`vector` 提供了如下几种 [迭代器](#)

1. `begin()/cbegin()`

返回指向首元素的迭代器, 其中 `*begin = front`。

2. `end()/cend()`

返回指向数组尾端占位符的迭代器, 注意是没有元素的。

3. `rbegin()/crbegin()`

返回指向逆向数组的首元素的逆向迭代器, 可以理解为正向容器的末元素。

4. `rend()/crend()`

返回指向逆向数组末元素后一位置的迭代器，对应容器首的前一个位置，没有元素。

以上列出的迭代器中，含有字符 `c` 的为只读迭代器，你 cannot 通过只读迭代器去修改 `vector` 中的元素的值。如果一个 `vector` 本身就是只读的，那么它的一般迭代器和只读迭代器完全等价。只读迭代器自 C++11 开始支持。

长度和容量

`vector` 有以下几个与容器长度和容量相关的函数。注意，`vector` 的长度 (size) 指有效元素数量，而容量 (capacity) 指其实际分配的内存长度，相关细节请参见后文的实现细节介绍。

与长度相关：

- `empty()` 返回一个 `bool` 值，即 `v.begin() == v.end()`，`true` 为空，`false` 为非空。
- `size()` 返回容器长度（元素数量），即 `std::distance(v.begin(), v.end())`。
- `resize()` 改变 `vector` 的长度，多退少补。补充元素可以由参数指定。
- `max_size()` 返回容器的最大可能长度。

与容量相关：

- `reserve()` 使得 `vector` 预留一定的内存空间，避免不必要的内存拷贝。
- `capacity()` 返回容器的容量，即不发生拷贝的情况下容器的长度上限。
- `shrink_to_fit()` 使得 `vector` 的容量与长度一致，多退但不会少。

元素增删及修改

- `clear()` 清除所有元素
- `insert()` 支持在某个迭代器位置插入元素、可以插入多个。**复杂度与 `pos` 距离末尾长度成线性而非常数的**
- `erase()` 删除某个迭代器或者区间的元素，返回最后被删除的迭代器。复杂度与 `insert` 一致。
- `push_back()` 在末尾插入一个元素，均摊复杂度为 **常数**，最坏为线性复杂度。
- `pop_back()` 删除末尾元素，常数复杂度。
- `swap()` 与另一个容器进行交换，此操作是 **常数复杂度** 而非线性的。

`vector` 的实现细节

`vector` 的底层其实仍然是定长数组，它能够实现动态扩容的原因是增加了避免数量溢出的操作。首先需要指明的是 `vector` 中元素的数量（长度）与它已分配内存最多能包含元素的数量（容量）是不一致的，`vector` 会分开存储这两个量。当向 `vector` 中添加元素时，如发现，那么容器会分配一个尺寸为 的数组，然后将旧数据从原本的位置拷贝到新的数组中，再将原来的内存释放。尽管这个操作的渐进复杂度是，但是可以证明其均摊复杂度为。而在末尾删除元素和访问元素则都仍然是 的开销。因此，只要对 `vector` 的尺寸估计得当并善用 `resize()` 和 `reserve()`，就能使得 `vector` 的效率与定长数组不会有太大差距。

`vector<LTbool>`

标准库特别提供了对 `bool` 的 `vector` 特化，每个「`bool`」只占 1 bit，且支持动态增长。但是其 `operator[]` 的返回值的类型不是 `bool&` 而是 `vector<LTbool>::reference`。因此，使用 `vector<LTbool>` 使需谨慎，可以考虑使用 `deque<LTbool>` 或 `vector<LTchar>` 替代。而如果你需要节省空间，请直接使用 [bitset](#)。

`array(C++11)`

`std::array` 是 STL 提供的 **内存连续的、固定长度** 的数组数据结构。其本质是对原生数组的直接封装。

为什么要用 `array`

`array` 实际上是 STL 对数组的封装。它相比 `vector` 牺牲了动态扩容的特性，但是换来了与原生数组几乎一致的性能（在开满优化的前提下）。因此如果能使用 C++11 特性的情况下，能够使用原生数组的地方几乎都可以直接把定长数组都换成 `array`，而动态分配的数组可以替换为 `vector`。

成员函数

隐式定义的成员函数

函数	作用
<code>operator=</code>	以来自另一 <code>array</code> 的每个元素重写 <code>array</code> 的对应元素

元素访问

函数	作用
<code>at</code>	访问指定的元素，同时进行越界检查
<code>operator[]</code>	访问指定的元素，不 进行越界检查
<code>front</code>	访问第一个元素
<code>back</code>	访问最后一个元素
<code>data</code>	返回指向内存中数组第一个元素的指针

`at` 若遇 `pos >= size()` 的情况会抛出 `std::out_of_range`。

容量

函数	作用
<code>empty</code>	检查容器是否为空
<code>size</code>	返回容纳的元素数
<code>max_size</code>	返回可容纳的最大元素数

由于每个 `array` 都是固定大小容器，`size()` 返回的值等于 `max_size()` 返回的值。

操作

函数	作用
<code>fill</code>	以指定值填充容器
<code>swap</code>	交换内容

注意，交换两个 `array` 是 的，而非与常规 STL 容器一样为 。

非成员函数

函数	作用
<code>operator==</code>	等 按照字典序比较 <code>array</code> 中的值
<code>std::get</code>	访问 <code>array</code> 的一个元素
<code>std::swap</code>	特化的 <code>std::swap</code> 算法

下面是一个 `array` 的使用示例：

```

1 // 1. 创建空array, 长度为3; 常数复杂度
2 std::array<int, 3> v0;
3 // 2. 用指定常数创建array; 常数复杂度
4 std::array<int, 3> v1{1, 2, 3};
5
6 v0.fill(1); // 填充数组
7
8 // 访问数组
9 for (int i = 0; i != arr.size(); ++i) cout << arr[i] << " ";

```

deque

`std::deque` 是 STL 提供的 [双端队列](#) 数据结构。能够提供线性复杂度的插入和删除，以及常数复杂度的随机访问。

deque 的使用方法

以下介绍常用用法，详细内容 [请参见 C++ 文档](#)。`deque` 的迭代器函数与 `vector` 相同，因此不作详细介绍。

构造函数

参见如下代码（假设你已经 `using` 了 `std` 命名空间相关类型）：

```

1 // 1. 定义一个int类型的空双端队列 v0
2 deque<int> v0;
3 // 2. 定义一个int类型的双端队列 v1, 并设置初始大小为10; 线性复杂度
4 deque<int> v1(10);
5 // 3. 定义一个int类型的双端队列 v2, 并初始化为10个1; 线性复杂度
6 deque<int> v2(10, 1);
7 // 4. 复制已有的双端队列 v1; 线性复杂度
8 deque<int> v3(v1);
9 // 5. 创建一个v2的拷贝deque v4, 其内容是v4[0]至v4[2]; 线性复杂度
10 deque<int> v4(v2.begin(), v2.begin() + 3);
11 // 6. 移动v2到新创建的deque v5, 不发生拷贝; 常数复杂度; 需要 C++11
12 deque<int> v5(std::move(v2));

```

元素访问

与 `vector` 一致，但无法访问底层内存。其高效的元素访问速度可参考实现细节部分。

- `at()` 返回容器中指定位置元素的引用，执行越界检查，**常数复杂度**。
- `operator[]` 返回容器中指定位置元素的引用。不执行越界检查，**常数复杂度**。
- `front()` 返回首元素的引用。
- `back()` 返回末尾元素的引用。

迭代器

与 `vector` 一致。

长度

与 `vector` 一致，但是没有 `reserve()` 和 `capacity()` 函数。（仍然有 `shrink_to_fit()` 函数）

元素增删及修改

与 `vector` 一致，并额外有向队列头部增加元素的函数。

- `clear()` 清除所有元素
- `insert()` 支持在某个迭代器位置插入元素、可以插入多个。复杂度与 `pos` 与两端距离较小者成线性。
- `erase()` 删除某个迭代器或者区间的元素，返回最后被删除的迭代器。复杂度与 `insert` 一致。
- `push_front()` 在头部插入一个元素，常数复杂度。
- `pop_front()` 删除头部元素，常数复杂度。
- `push_back()` 在末尾插入一个元素，常数复杂度。
- `pop_back()` 删除末尾元素，常数复杂度。
- `swap()` 与另一个容器进行交换，此操作是常数复杂度而非线性的。

deque 的实现细节

`deque` 通常的底层实现是多个不连续的缓冲区，而缓冲区中的内存是连续的。而每个缓冲区还会记录首指针和尾指针，用来标记有效数据的区间。当一个缓冲区填满之后便会在之前或者之后分配新的缓冲区来存储更多的数据。更详细的说明可以参考 [STL 源码剖析——deque 的实现原理和使用方法详解](#)。

list

`std::list` 是 STL 提供的 [双向链表](#) 数据结构。能够提供线性复杂度的随机访问，以及常数复杂度的插入和删除。

list 的使用方法

`list` 的使用方法与 `deque` 基本相同，但是增删操作和访问的复杂度不同。详细内容 [请参见 C++ 文档](#)。`list` 的迭代器、长度、元素增删及修改相关的函数与 `deque` 相同，因此不作详细介绍。

元素访问

由于 `list` 的实现是链表，因此它不提供随机访问的接口。若需要访问中间元素，则需要使用迭代器。

- `front()` 返回首元素的引用。
- `back()` 返回末尾元素的引用。

操作

`list` 类型还提供了一些针对其特性实现的 STL 算法函数。由于这些算法需要 [随机访问迭代器](#)，因此 `list` 提供了特别的实现以便于使用。这些算法有 `splice()`、`remove()`、`sort()`、`unique()`、`merge()` 等。

forward_list (C++11)

`std::forward_list` 是 STL 提供的 [单向链表](#) 数据结构，相比于 `std::list` 减小了空间开销。

forward_list 的使用方法

`forward_list` 的使用方法与 `list` 几乎一致，但是迭代器只有单向的，因此其具体用法不作详细介绍。详细内容 [请参见 C++ 文档](#)

本页面最近更新：2023/12/16 23:06:11，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[cmpute](#)、[Xeonacid](#)、[1475505](#)、[CCXXI](#)、[ChenZ01](#)、[Enter-tainer](#)、[greyqz](#)、[i-Yirannn](#)、[lr1d](#)、[mgt](#)、[MingqiHuang](#)、[ouuan](#)、[StudyingFather](#)、[Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用

